



Thin-Airfoil Theory Solver for NACA 4-Digit Airfoils

Part II: Software Implementation, Automated XFOIL Coupling, and Validation

Ventura, César Carlos

Departamento de Física, UNS
Av. Alem 1253, (8000) Bahía Blanca, Argentina
E-mail: cesarventura.phys@gmail.com

Contents

1	Introduction	2
2	Recap of the Governing Equations	2
3	Software Implementation	3
3.1	Program Modules	3
3.2	Numerical Considerations: Singularity Handling	4
4	Automated XFOIL Coupling	5
4.1	XFOIL as a Scriptable Subprocess	5
4.1.1	Polar Sweep	5
4.1.2	Surface-Pressure Snapshot	6
5	Validation Against XFOIL	6
5.1	Lift Curve (Polar) Comparison	7
5.2	Chordwise Loading Comparison	8
5.3	Summary of the Extended Validation	9
6	Conclusions	10

Abstract

This document presents Part II of a two-part project on the development and validation of a thin-airfoil theory python solver for NACA four-digit series airfoils. Building upon the previous program, this updated version automates the execution of XFOIL’s panel method to enable a direct comparison between the two aerodynamic models. The solver computes the vortex-sheet strength $\gamma(\theta)$ via an analytical Fourier-series solution of the thin-airfoil integral equation, from which the lift coefficient C_l and chordwise loading $\Delta C_p(x/c)$ are obtained. Two extensions are introduced: the systematic evaluation of C_l as a continuous function of angle of attack over a prescribed range, and the automated generation of XFOIL reference data that replicates the interactive console sequence without manual intervention. Validation is performed against XFOIL inviscid results for both the cambered NACA 2412 and symmetric NACA 0012 airfoils across lift-curve sweeps and chordwise loading distributions at $\alpha = 5^\circ$. The results demonstrate qualitative agreement, with the expected systematic offset attributable to the zero-thickness assumption of thin-airfoil theory. The solver accepts any valid NACA four-digit code and generates comparison figures automatically.

1 Introduction

Part I of this project [1] established the theoretical foundation and the a thin-airfoil python solver for NACA four-digit airfoils, deriving analytical Fourier coefficients for the vortex-sheet strength, which in turn calculates the lift coefficient, pressure distribution field, and the streamline field. This document, Part II, focuses on the validation framework and comparison with the industry-standard XFOIL [2] panel-method program. The updated solver is reorganized as a modular Python package with two additions: automated generation of lift-curve polars over a range of angles of attack, and computation of the chordwise loading $\Delta C_p(x/c)$. An automated XFOIL coupling driver is also introduced to generate reference data without manual intervention.

Validation is performed against XFOIL inviscid results for both the cambered NACA 2412 and symmetric NACA 0012 airfoils, comparing lift coefficients across a polar sweep and chordwise loading distributions at $\alpha = 5^\circ$. The results show the expected behavior of thin-airfoil theory and demonstrate the correctness of the implementation.

2 Recap of the Governing Equations

The vortex-sheet strength along the chord is represented as the Fourier series

$$\gamma(\theta) = 2V_\infty \left[A_0 \frac{1 + \cos \theta}{\sin \theta} + \sum_{n=1}^N A_n \sin(n\theta) \right], \quad \theta \in [0, \pi], \quad (1)$$

with $x(\theta) = \frac{c}{2}(1 - \cos \theta)$, where the coefficients $A_0, \dots, A_N$ are obtained analytically for the NACA four-digit camber slope $dy/dx(\theta)$ as

$$A_0 = \alpha - \frac{1}{\pi} \int_0^\pi \frac{dy}{dx} d\theta, \quad A_n = \frac{2}{\pi} \int_0^\pi \frac{dy}{dx} \cos(n\theta) d\theta, \quad n \geq 1. \quad (2)$$

The lift coefficient follows directly from the first two coefficients,

$$c_l = \pi (2A_0 + A_1), \quad \frac{dc_l}{d\alpha} = 2\pi, \quad (3)$$

predicting a lift-curve slope of exactly 2π per radian independent of camber, camber enters only through the zero-lift angle. The chordwise loading (pressure jump across the sheet) is

$$\Delta C_p(\theta) \equiv C_{p,\text{lower}} - C_{p,\text{upper}} = \frac{2\gamma(\theta)}{V_\infty}. \quad (4)$$

Equations (1) to (4) are the three quantities this report validates against XFOIL.

3 Software Implementation

The solver documented in this section is an updated and extended implementation of the thin-airfoil Python code introduced in Part I. Retaining the original capability to compute pressure fields and streamline patterns for any specified NACA four-digit airfoil, this version introduces two extensions. First, it evaluates the lift coefficient C_l as a continuous function of the angle of attack by sweeping α over a prescribed range. Second, it leverages the computed vortex-sheet strength $\gamma(\theta)$ from Eq. (1) to directly obtain the chordwise loading ΔC_p as a function of the normalized chordwise position x/c , thereby characterizing the surface-pressure jump across the airfoil. The program now also automates the execution of XFOIL panel-method calculations for the same airfoil geometry, parsing the resulting polar and surface-pressure data to generate direct graphical comparisons between both methods.

3.1 Program Modules

- **config.py** - Specifies the user-defined parameters: airfoil geometry (M , P , C), flow conditions (α , V_∞), the Fourier truncation order N , the visualization grid (X/Y bounds and resolution), and streamline-integration settings. When the solver is run interactively, these defaults are overridden per call rather than edited in place, so a single process can evaluate several airfoils in sequence.
- **fourier_coefficients.py** - symbolic derivation of Eq. (2) using SymPy: the piecewise camber slope is defined symbolically, integrated analytically over $[0, \theta_p]$ and $[\theta_p, \pi]$, simplified using $\cos \theta_p = 1 - 2p$, and compiled into fast numerical functions via `sympy.lambdify`, evaluated once at import time. The symmetric case ($m = 0$) is special-cased to return $A_0 = \alpha$, $A_n = 0$ directly, avoiding an unnecessary symbolic evaluation.
- **vortex.py** - Eq. (1) for $\gamma(\theta)$, together with a numerically safe variant $\gamma(\theta) \sin \theta$ that

removes the leading-edge $(1/\sin\theta)$ singularity, plus the total circulation Γ and the lift coefficient c_l of Eq. (3).

- `velocity.py` - Handles the coordinate transform $x(\theta)/\theta(x)$, the Biot-Savart induced-velocity integrals, the total velocity and pointwise pressure coefficient, and a vectorized `compute_field_grid()` used to evaluate C_p , u , v over an entire visualization mesh.
- `streamlines.py` - integrates streamline trajectories $dx/dt = u$ and $dy/dt = v$.
- `visualize.py` - the camber-line geometry helper `y_camber()`, the filled pressure-contour plot, and a stream-function heatmap that reconstructs $\psi(x, y)$ from the velocity grid by direct marching integration rather than by tracing individual streamlines.
- `xfoil_parser.py` - Parsers for XFOIL output: `read_xfoil_polar()` locates the alpha/CL column header by regular-expression pattern matching and returns a `pandas.DataFrame`; `read_xfoil_cpwr()` reads a surface-pressure dump and separates the upper/lower surfaces by the sign of the local y -coordinate rather than by file order.
- `xfoil_runner.py` - Parses a NACA four-digit code into (M, P, T) and drives XFOIL as a subprocess to produce the two reference datasets the comparison needs, without any manual interaction.
- `generate_polar.py` / `generate_loading.py` - reusable, parameterized drivers (`compute_polar()` and `compute_loading()`) that sweep α or evaluate Eq. (4) at a single α , respectively, and overlay the result with the corresponding parsed XFOIL data. Both accept M , P , and an airfoil label as arguments, so they are no longer tied to a single hard-coded airfoil.
- `main.py` - the original top-level driver for a single.
- `interactive_main.py` - new for part II. A prompt entry point that asks the user for a NACA four-digit code and a small set of run parameters.

3.2 Numerical Considerations: Singularity Handling

Both the leading-edge $(1/\sin\theta)$ term in Eq. (1) and the on-chord induced velocity are singularities. The former is handled analytically by working with the regular product $\gamma(\theta)\sin\theta$, computed directly by `gamma_theta_safe()` and passed to the quadrature integrand in `velocity.py`, so that no singularity is ever numerically encountered. The latter is handled by perturbing field points that fall exactly on $y = 0$ by 10^{-14} before

quadrature, and by masking a small disk of radius `LE_MASK_RADIUS` around the leading edge in all field plots and in the loading comparison (`MASK_FRACTION`).

4 Automated XFOIL Coupling

A significant part of this stage of the project was making the XFOIL comparison itself reproducible without manual, per-airfoil interaction with XFOIL’s text-based menu system. This section documents the design of that automation.

4.1 XFOIL as a Scriptable Subprocess

XFOIL is an interactive, menu-driven console program. However, because it reads its menu selections from standard input, the exact sequence of keystrokes a user would type interactively can be written to a text buffer and piped to XFOIL’s `stdin`. The automation driver `xfoil_runner.py` builds this buffer for two distinct XFOIL sessions, run as separate subprocess invocations.

4.1.1 Polar Sweep

The first session generates a lift curve over a specified angle of attack range. The command follows the sequence:

1. Load the airfoil geometry: `NACA <airfoil code>`
2. Enter the geometry design menu and immediately exit: `GDES` , followed by a blank line
3. Enter the operation menu in inviscid mode: `OPERi`
4. Set the iteration limit to ensure convergence: `ITER 400`
5. Begin polar accumulation: `PACC`
6. Specify the output filename: `polar_<naca>.txt`
7. Sweep the angle of attack: `ASEQ <alpha_min> <alpha_max> <step>`
8. Exit XFOIL: `QUIT`

Listing 1 shows a concrete example for a NACA 2412 airfoil from -5° to 15° in 1° steps:

Listing 1: Generated XFOIL command buffer for the polar sweep.

```
NACA 2412
GDES
OPERi
```

```

ITER 400
PACC
polar_2412.txt

ASEQ -5 15 1

QUIT

```

4.1.2 Surface-Pressure Snapshot

The second session generates the chordwise pressure distribution at a single angle of attack, using a new XFOIL. The command sequence is:

1. Load the airfoil geometry: `NACA <airfoil code>`
2. Enter and immediately exit `GDES`
3. Enter inviscid operating mode: `OPERi`
4. Set the iteration limit: `ITER 400`
5. Set the angle of attack: `A <alpha>`
6. Write the surface-pressure distribution: `CPWR cp_<naca>_a<alpha>.txt`
7. Exit XFOIL: `QUIT`

Listing 2 illustrates the buffer for the same NACA 2412 airfoil at $\alpha = 5^\circ$:

Listing 2: Generated XFOIL command buffer for the surface-pressure snapshot.

```

NACA 2412
GDES

OPERi
ITER 400
A 5
CPWR cp_2412_a5.txt

QUIT

```

In both scripts, the NACA code and α values are interpolated from user input at runtime, making the driver applicable to any four-digit series airfoil and any analysis condition.

5 Validation Against XFOIL

To demonstrate that the solver generalizes correctly across camber, the validation is performed for two representative airfoils: the cambered NACA 2412 ($m = 0.02$, $p = 0.4$, $t = 0.12$) and the symmetric NACA 0012 ($m = 0$, $p = 0$, $t = 0.12$). Both cases are generated through `interactive_main.py`, with XFOIL reference data produced automatically by the driver.

5.1 Lift Curve (Polar) Comparison

`generate_polar.py` sweeps α from -5° to 15° in 1° increments, recomputing $A(\alpha)$ and $c_l(\alpha)$ from Eqs. (2)-(3) at each step, and overlays the result with the XFOIL polar generated automatically for the same airfoil. Figures 1 and 2 show the resulting comparisons for the NACA 2412 and NACA 0012 airfoils, respectively.

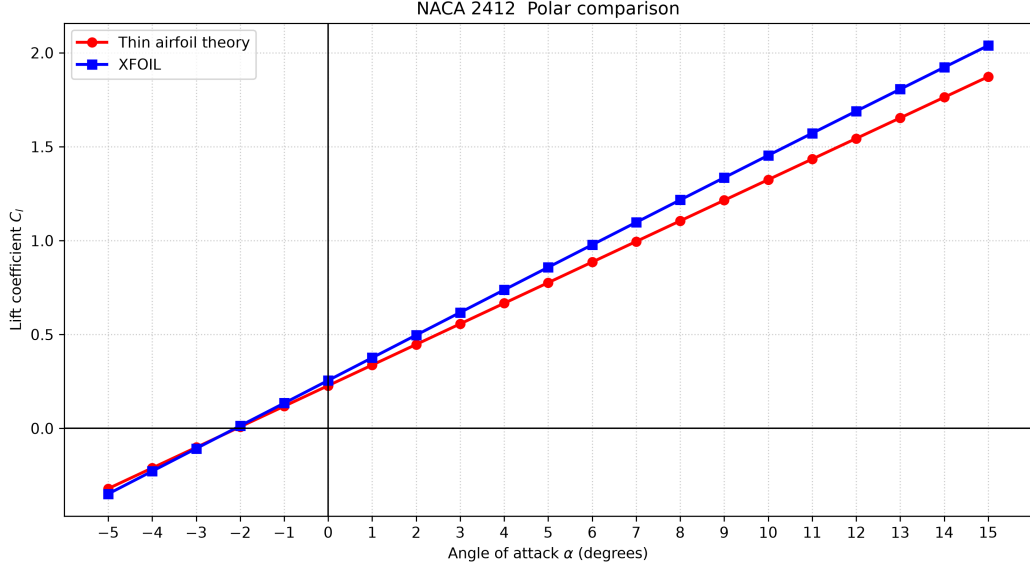


Figure 1: Lift coefficient vs. angle of attack: thin airfoil theory (constant 2π slope) vs. XFOIL panel-method polar, NACA 2412.

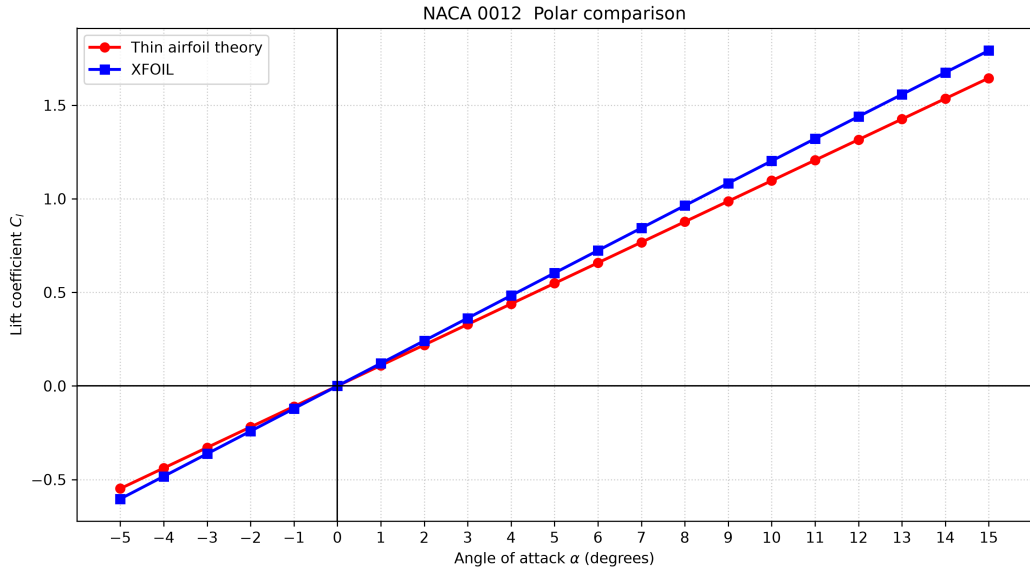


Figure 2: Lift coefficient vs. angle of attack: thin airfoil theory (constant 2π slope) vs. XFOIL panel-method polar, NACA 0012.

Because Eq. (3) predicts a lift slope of exactly 2π per radian independent of camber, the thin-airfoil polar is a straight line whose only camber-dependent feature is its horizontal offset (the zero-lift angle α_{L0}). For NACA 2412, the XFOIL polar exhibits a similarly

linear trend but with a slightly higher slope and higher C_l at every angle of attack. This is expected, XFOIL’s panel method retains the true airfoil thickness distribution, and finite thickness increases the effective camber and local curvature seen by the flow near the leading edge. For the symmetric NACA 0012 case, the thin-airfoil polar passes through the origin ($\alpha_{L0} = 0$), and the XFOIL result again shows a slightly steeper slope due to thickness effects. In both cases, the two curves agree in slope and zero-lift angle to within a small offset, confirming that the Fourier solution behind Eq. (3) is implemented correctly across the full angle range.

5.2 Chordwise Loading Comparison

`generate_loading.py` evaluates the chordwise loading $\Delta C_p(x/c)$ of Eq. (4) on a fine θ -grid, masks a small region near the leading and trailing edges (where both the thin-airfoil singularity and the linear interpolation of the discrete XFOIL data become unreliable), and compares the result with ΔC_p obtained from XFOIL’s surface-pressure file by separately interpolating the upper and lower surface $C_p(x)$ curves and subtracting them. Figures 3 and 4 show the results at $\alpha = 5^\circ$ for both airfoils, using the automatically generated CPWR data as the surface pressure reference.

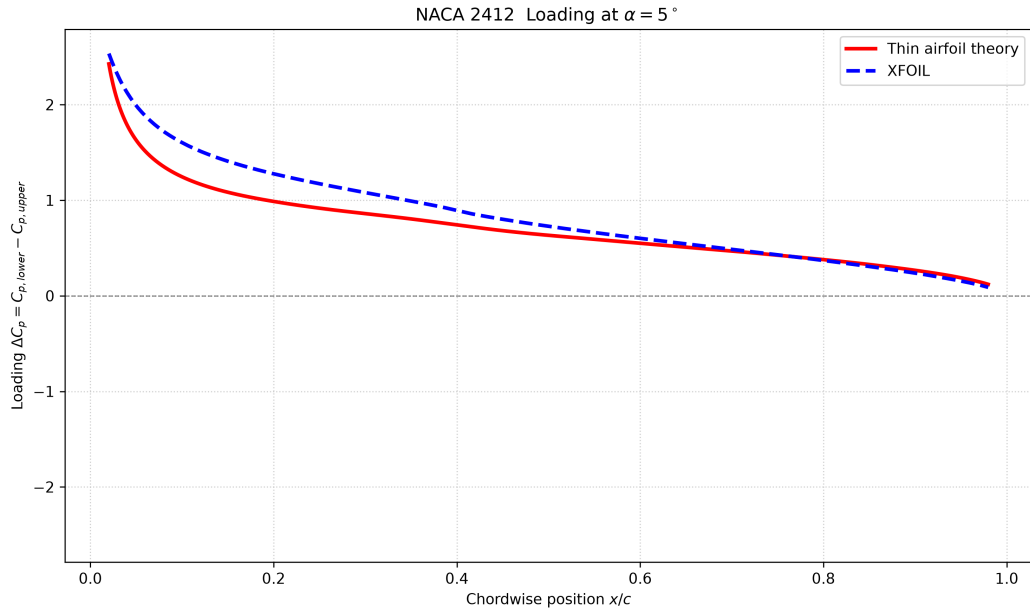


Figure 3: Chordwise loading $\Delta C_p = C_{p,\text{lower}} - C_{p,\text{upper}}$ vs. x/c : thin airfoil theory vs. XFOIL, NACA 2412 at $\alpha = 5^\circ$.

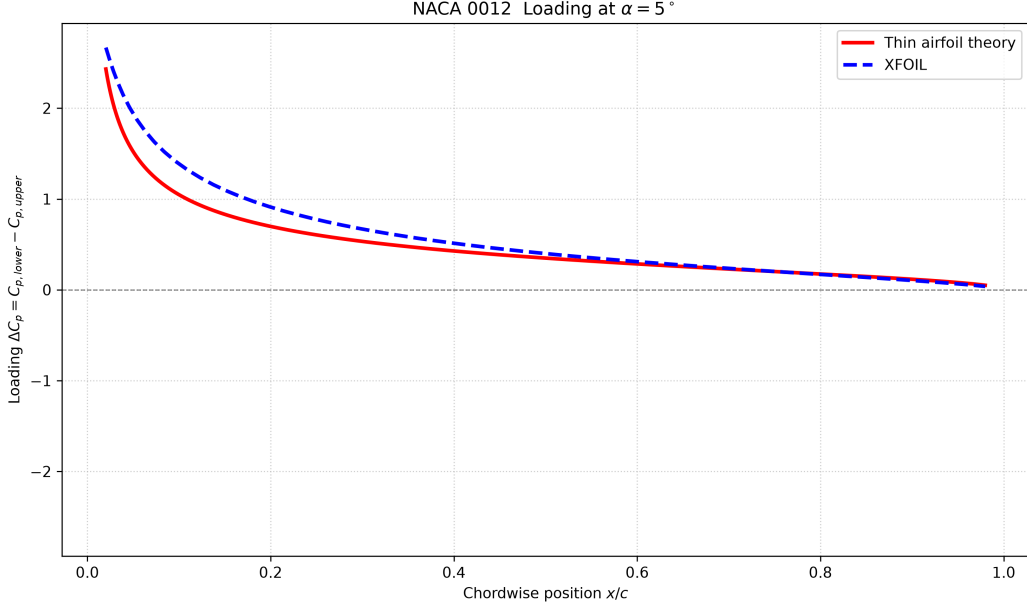


Figure 4: Chordwise loading $\Delta C_p = C_{p,\text{lower}} - C_{p,\text{upper}}$ vs. x/c : thin airfoil theory vs. XFOIL, NACA 0012 at $\alpha = 5^\circ$.

The thin-airfoil loading correctly reproduces the qualitative shape of the XFOIL loading distribution for both airfoils: a large loading peak near the leading edge that decays smoothly toward the trailing edge. As with the polar comparison, the thin-airfoil curve systematically underpredicts the magnitude of the loading relative to XFOIL, with the largest discrepancy near the leading edge.

5.3 Summary of the Extended Validation

Table 1 summarizes the two physical validation categories added in this part, for both airfoils.

Airfoil	Quantity compared	Outcome
NACA 2412	$c_l(\alpha)$, $\alpha \in [-5^\circ, 15^\circ]$	Correct slope trend, small offset
NACA 0012	$c_l(\alpha)$, $\alpha \in [-5^\circ, 15^\circ]$	Correct slope trend, small offset
NACA 2412	$\Delta C_p(x/c)$ at $\alpha = 5^\circ$	Correct shape, underpredicted peak
NACA 0012	$\Delta C_p(x/c)$ at $\alpha = 5^\circ$	Correct shape, underpredicted peak

Table 1: Summary of the validation categories and their outcomes.

The agreement between the thin-airfoil solver and XFOIL across both airfoils and both validation metrics confirms that the implementation correctly captures the governing inviscid physics for both cambered and symmetric geometries. The systematic discrepancies are well understood and arise from the inherent limitations of the zero-thickness model, not from errors in the numerical implementation or the Fourier-coefficient derivation.

6 Conclusions

This second part of the project has documented the thin-airfoil solver developed in Part I as it exists in working software form, automated its comparison against XFOIL so that neither reference dataset needs to be prepared by hand, and extended its validation from a single lift coefficient check to a full lift-curve sweep and a full chordwise loading comparison.

Both extended physical comparisons in Section 5 show the behavior expected of a linear, zero-thickness theory: correct qualitative shape and correct trends with angle of attack and chordwise position, together with a systematic, physically explainable underprediction of magnitude relative to the finite-thickness panel-method solution. The agreement across both the cambered NACA 2412 and symmetric NACA 0012 airfoils confirms that the implementation correctly captures the governing inviscid physics for both geometries. The summary table in Section 5 provides a concise overview of the validation categories and their outcomes, demonstrating that the solver meets its design objectives.

References

- [1] Ventura, C. (2026). Thin-Airfoil Theory Solver for NACA 4-Digit Airfoils, Part I: Coefficient of Lift, Streamline and Pressure Fields [Computer software]. GitHub. <https://github.com/cesarventura-phys/Streamline-and-Pressure-Fields-for-NACA-Four-Digit-Series>
- [2] Drela, M. (2013). XFOIL: Subsonic airfoil development system (Version 6.99) [Computer software]. MIT. <https://web.mit.edu/drela/Public/web/xfoil/>